

UNIVERSITY OF BUEA

P.O. Box 63,
Molyko - Buea, South West Region,
Cameroon
Tel: (+237) 233 32 21 34 / 233 32 26
90 | Fax: (+237) 233 32 22 72



REPUBLIC OF CAMEROON

PEACE-WORK-FATHERLAND

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

**DEVELOPMENT OF A ROAD SIGN AND ROAD STATE
MOBILE NOTIFICATION APPLICATION**

BY: GROUP 5

NAMES	MATRICULE
DJUMATCHE NANKAPMARC-OLIVIER	FE22A189
SOYANG THIERRY NGONG	FE22A298
TENDONG BRAIN NKENGAFAC	FE22A314
TIDDING RAMSEY BINDA	FE22A316
TIFUH PERCILIA NJI	FE22A317

Option: Software Engineering

COURSE INSTRUCTOR:

DR. VALERY NKEMENI
University Of Buea

2024/2025 Academic Year

**DEVELOPMENT OF A ROAD SIGN AND ROAD STATE MOBILE NOTIFICATION
APPLICATION**

NAMES	MATRICULE
DJUMATCHE NANKAPMARC-OLIVIER	FE22A189
SOYANG THIERRY NGONG	FE22A298
TENDONG BRAIN NKENGAFAC	FE22A314
TIDDING RAMSEY BINDA	FE22A316
TIFUH PERCILIA NJI	FE22A317

2024/2025 Academic Year

**Department of Computer Engineering
Faculty of Engineering and Technology
University of Buea**

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our lecturer Dr. Nkemeni Valery for his expert guidance, patience, and invaluable feedback throughout this project. His insight helped shaped this work into its current form

We extend our sincere thanks to the Department of Computer Engineering, Faculty of Engineering and Technology, University of Buea, for providing us with the necessary academic framework, resources, and a conducive learning environment.

Our appreciation also goes to all the road users in Cameroon who participated in our survey and interviews. Their candid feedback and perspectives were crucial in identifying the core problems and validating the functional requirements of this application.

ABSTRACT

This report presents a comprehensive overview of the "Road Sign and Road State Mobile Notification Application," detailing the requirements gathering, analysis, system modeling, design, database implementation, and backend API development phases. The aim of this application is to enhance road safety in Cameroon by providing real-time hazard notifications and accessible road sign education to various road users. This document adheres to the guidelines for writing final year project reports as prescribed by the Department of Computer Engineering, Faculty of Engineering and Technology, University of Buea.

The project addresses significant road safety challenges identified through an onsite survey conducted from April 11 to April 14, 2025, involving 100 diverse road users in Cameroon. Key findings revealed pervasive issues such as potholes (78%), traffic congestion (70%), and missing or unclear road signs (34%). The survey also highlighted a strong demand for features like voice alerts for traffic and accidents (76% prioritization), a comprehensive road sign directory (64% demand), and a high willingness among users to report road incidents (86%).

The application, internally named WayWatch for its backend system, is designed to serve a broad spectrum of users, including private car drivers, bus drivers, taxi drivers, traffic officers, motorbike riders, driving instructors, and pedestrians. It integrates real-time data collection, crowd-sourced reporting with a verification system, and location-based services to deliver critical road information, navigation assistance, and timely safety alerts. The report elaborates on the system's architecture, including its database design utilizing PostgreSQL with PostGIS for geospatial data, and a Node.js/Express.js backend for robust API implementation covering user authentication, location services, road sign management, and alert distribution.

A detailed literature review and gap analysis demonstrate that existing navigation and traffic applications fall short in providing comprehensive road sign education, verified crowd-sourced reporting, automatic emergency detection, and multilingual road safety content, particularly relevant for rural areas with limited connectivity. The proposed application aims to fill these critical market gaps.

The design phase utilizes UML diagrams (Context, Dataflow, Use Case, Sequence, Class, Deployment) to visually represent the system's structure and behavior, emphasizing scalability, maintainability, and security. The implementation section details the backend API and database, ensuring efficient and secure operations. Finally, the report discusses testing and validation methodologies and outlines future work, including wireframe design, API integrations, and further user testing to refine the application. By providing a solution that balances safety, education, and accessibility, this application is poised to significantly contribute to reducing accidents and fostering safer roads in Cameroon.

TABLE OF CONTENTS

Acknowledgement	3
Abstract	4
List of Abbreviation.....	7
 CHAPTER 1. GENERAL INTRODUCTION	 8
1.1 Background	8
1.2 Problem Statement	8
1.3 Objectives of the Study	8
1.4 Scope of the Project	8
1.5 Significance of the Project	10
1.6 Organization of the Report	10
 CHAPTER 2. LITERATURE REVIEW	 12
2.1 Various Existing Systems Reviewed	12
2.2 Detailed Gap Analysis	13
2.2.1 Google Maps	13
2.2.2 511 Apps (State Versions)	14
2.2.3 Sygic	14
2.2.4 FHWA Mobile Apps	15
2.2.5 PaveScope	15
2.3 Summary of Critical Market Gaps	16
 CHAPTER 3. ANALYSIS AND DESIGN	 18
3.1 Requirements Analysis	18
3.1.1 Objectives of Requirements Analysis	19
3.1.2 Methodology for Requirements Analysis	19
3.1.3 Functional Requirements	19
3.1.4 Non-Functional Requirements	20
3.1.5 System Requirements	21
3.1.6 User Requirements	21
3.1.7 Prioritized Requirements	22
3.1.8 Legal and Regulatory Requirements	22
3.1.9 Acceptance Test Plan	23
3.2 System Design and Modelling	23
3.2.1 Purpose and Scope	23
3.2.2 System Design Overview	23
3.2.2.1 System Architecture	23
3.2.2.2 Key Features	23
3.2.3 UML Diagrams	24
3.2.3.1 Context Diagram	24
3.2.3.2 Dataflow Diagram	24
3.2.3.3 Use Case Diagram	25
3.2.3.4 Sequence Diagrams	25

3.2.3.5 Class Diagram	30
3.2.3.6 Deployment Diagram	31
3.2.4 Design Analysis	32
3.2.4.1 Structural Analysis	32
3.2.4.2 Behavioral Analysis	32
3.2.4.3 Design Strengths	32
3.2.4.4 Implementation Considerations	33
3.2.5 Conclusion of System Modelling and Design	33
3.3 UI Design	33
3.3.1 Wireframes and Mockups.....	34
3.3.2 Design Specifications.....	36
3.3.3 UI Components and Patterns.....	37
3.3.4 Accessibility.....	37
CHAPTER 4. IMPLEMENTATION AND RESULTS	38
4.1 Front End Implementation.....	38
4.2 Database Design and Implementation	40
4.2.1 System Overview	40
4.2.2 Database Design and Implementation	40
4.2.2.1 Database Architecture	40
4.2.2.2 Schema Design	41
4.2.2.3 Table Relationships	42
4.2.2.4 Implementation Details	42
4.2.3 Backend API Implementation	43
4.2.3.1 User Authentication System	43
4.2.3.2 OTP Verification	44
4.2.3.3 Login Authentication	44
4.2.3.4 Location Services	45
4.2.3.5 Road Sign Management	45
4.2.3.6 Crowd Reporting System	46
4.2.3.7 Alert Distribution System	46
4.2.3.8 User Profile Management	47
4.3 Testing and Validation	48
4.3.1 Database Testing	48
4.3.2 API Testing Methodology	48
CHAPTER 5. CONCLUSION AND FURTHER WORKS	49
5.1 Key Achievements	49
5.2 Future Work	49
REFERENCES	51

LIST OF ABBREVIATIONS

- **API:** Application Programming Interface
- **B.Eng.:** Bachelor of Engineering
- **DFD:** Data Flow Diagram
- **FHWA:** Federal Highway Administration
- **GPS:** Global Positioning System
- **GDPR:** General Data Protection Regulation
- **JWT:** JSON Web Token
- **OTP:** One-Time Password
- **UML:** Unified Modeling Language
- **UI:** User Interface
- **WCAG:** Web Content Accessibility Guidelines

CHAPTER 1. GENERAL INTRODUCTION

1.1 Background

Road safety and efficient traffic management are critical components of modern urban and rural infrastructure. In many regions, conventional static road signage, often subject to wear, damage, or poor visibility, coupled with delayed and unverified incident reporting mechanisms, contribute significantly to road accidents, traffic congestion, and overall navigational inefficiencies. The advent of mobile technology and ubiquitous internet connectivity presents an opportunity to revolutionize how road users interact with their environment and receive critical safety information. This project focuses on developing a mobile application designed to overcome these traditional limitations by providing real-time, dynamic road information and educational content.

1.2 Problem Statement

Despite advancements in navigation and communication technologies, road users in Cameroon continue to face significant challenges that compromise safety and efficiency. An onsite survey conducted from April 11 to April 14, 2025, involving 100 diverse road users, highlighted prevalent issues: 78% of respondents reported struggling with potholes, 70% faced chronic traffic congestion, and a notable 34% encountered missing or unclear road signs. Furthermore, reckless driving was identified as a concern by 50% of the respondents. These challenges lead to increased accident rates, longer travel times, economic losses, and a general sense of insecurity on the roads. Existing solutions often lack comprehensive, real-time, and localized information, fail to integrate educational components, or do not offer robust crowd-sourced reporting with verification mechanisms. There is a clear need for a holistic solution that actively addresses these multifaceted problems.

1.3 Objectives of the Study

The primary objective of this project is to design, develop, and implement a "Road Sign and Road State Mobile Notification Application" to enhance road safety and navigation efficiency in Cameroon.

Specific objectives include:

- To gather and analyze functional and non-functional requirements from diverse road users.
- To design a scalable and robust system architecture using UML diagrams.
- To implement a secure and efficient backend system, including database design and API development.
- To integrate real-time location-based services and crowd-sourced reporting features.
- To provide a comprehensive, interactive, and multilingual road sign educational directory.
- To develop a reliable alert distribution system for real-time hazard notifications.
- To validate the system's functionality, performance, and security through rigorous testing.

1.4 Scope of the Project

The scope of this project encompasses the development of the backend system and database for the "Road Sign and Road State Mobile Notification Application" (internally named WayWatch). This includes:

- **Requirements Gathering and Analysis:** Defining the needs based on user surveys and stakeholders.
- **System Design and Modeling:** Creating detailed architectural and behavioral designs using UML.
- **Database Design:** Designing a robust and scalable database schema, particularly focusing on geospatial data management using PostgreSQL with PostGIS.
- **Backend API Development:** Implementing a RESTful API using Node.js and Express.js to handle core functionalities such as user authentication, location tracking, road sign management, crowd reporting, and real-time alert distribution.
- **Security Features:** Implementing OTP verification, encrypted password storage, and JWT-based session management.
- **Testing:** Conducting database and API testing to ensure functionality, performance, and security.

This project focuses on the backend infrastructure and does not include the development of the client-side mobile application (frontend UI/UX), although its requirements heavily influence the backend design.

1.5 Significance of the Project

The "Road Sign and Road State Mobile Notification Application" holds significant importance for road safety and community well-being in Cameroon. By providing real-time, verified information on road conditions, accidents, and hazards, it aims to:

- **Reduce Road Accidents:** Proactive alerts can help drivers avoid dangerous situations, reducing the incidence of collisions.
- **Improve Traffic Flow:** Real-time information on congestion and alternative routes can optimize travel times and reduce frustration.
- **Enhance Road Sign Comprehension:** The interactive educational directory will improve knowledge of road signs, particularly for new drivers and in areas with unclear signage.
- **Empower Road Users:** The crowd-reporting feature allows users to actively contribute to road safety by reporting issues, fostering a sense of community responsibility.
- **Support Emergency Services:** Automatic distress signals and location sharing can facilitate quicker response times in emergencies.
- **Promote Data-Driven Road Management:** The collected data can provide valuable insights for traffic authorities and urban planners to improve infrastructure and policies.
- **Increase Accessibility:** Offline mode and multilingual support will ensure the application is beneficial even in rural areas with limited connectivity and diverse linguistic backgrounds.

Ultimately, this project seeks to create a safer, more informed, and efficient road environment for all in Cameroon.

1.6 Organization of the Report

This report is organized into five main chapters, followed by references and appendices.

- **Chapter 1: General Introduction** provides an overview of the project, including its background, problem statement, objectives, scope, and significance.
- **Chapter 2: Literature Review** critically examines existing road safety and navigation applications, highlighting their strengths and identifying key gaps that the proposed system aims to address.

- **Chapter 3: Analysis and Design** details the requirements gathering process, functional and non-functional requirements, and presents the system architecture and design using various UML diagrams, including the UI design.
- **Chapter 4: Implementation and Results** describes the database design, backend API development, front-end implementation, and the testing methodologies employed to validate the system.
- **Chapter 5: Conclusion and Further Works** summarizes the project's achievements, discusses its limitations, and outlines potential future enhancements and research directions.

CHAPTER 2. LITERATURE REVIEW

This section reviews existing systems that aim to provide navigation, traffic updates, and road condition information, and then presents a detailed gap analysis of these systems against the functional requirements of the proposed "Road Sign and Road State Mobile Notification Application."

2.1 Various Existing Systems Reviewed

This review considers a range of applications and platforms that offer various aspects of road information and navigation.

- **Google Maps:** This widely used application provides comprehensive real-time navigation with traffic updates, route optimization, and alternative route suggestions. It includes street view integration, allowing users to visualize routes before travel, and supports offline map downloads for navigation in areas with limited connectivity. Google Maps primarily focuses on efficient travel and general awareness of traffic conditions.
- **511 Apps (Various state versions):** These applications are typically developed by official state transportation departments. They serve as primary sources for real-time traffic cameras, road closure and construction alerts, and weather-related road conditions specific to a particular state or region. Their strength lies in providing official, government-issued information directly to the public.
- **Sygie:** A navigation application known for its robust offline capabilities, particularly for commercial vehicles. Sygie offers truck-specific routing, which takes into account vehicle dimensions and weight restrictions, and provides speed camera alerts to help drivers adhere to speed limits and avoid penalties. Its focus is on professional driving and specialized navigation needs.
- **FHWA Mobile Apps (Federal Highway Administration):** The Federal Highway Administration promotes and, in some cases, develops various state-specific transportation apps. These applications often provide information on construction zones, emergency road conditions, and sometimes basic traffic data. They are generally informational tools aimed at public awareness regarding large-scale road projects and incidents.
- **PaveScope:** This is a specialized professional tool designed for road surface condition assessment, pavement quality reporting, and infrastructure monitoring. PaveScope is used by civil engineers and road maintenance authorities to evaluate and manage road

networks, focusing on the physical state and integrity of roads rather than consumer-facing navigation or traffic alerts.

2.2 Detailed Gap Analysis

This section provides a detailed gap analysis for each existing system against the proposed functional requirements of the "Road Sign and Road State Mobile Notification Application." The analysis highlights areas where current solutions fall short, justifying the need for the new application.

2.2.1 Google Maps

Google Maps is a powerful navigation tool, but it has significant limitations concerning the specific requirements of the proposed application.

- **Major Gaps:**
 - ✓ **Road Sign Directory (Complete Gap):** Google Maps offers no built-in feature for identifying road signs, providing explanations, or offering educational content about them. It assumes user familiarity with signs.
 - ✓ **Image Recognition:** It cannot identify road signs through a device's camera for on-the-spot learning or assistance.
 - ✓ **Crowd Reporting Verification:** While it has basic user reporting for incidents, there is no sophisticated AI or moderator-based verification system to ensure the accuracy and reliability of crowd-sourced data.
 - ✓ **Reckless Driving Reports:** There is no specific, dedicated feature for users to report dangerous or reckless driving behavior.
 - ✓ **Weather-Road Integration:** It provides general weather information, but there is limited correlation between specific weather conditions (e.g., heavy rain, fog) and their direct impact on road hazards (e.g., slippery roads, reduced visibility in specific areas).
 - ✓ **Educational Content:** It lacks any form of learning modules, exam-preparation modes, or quizzes related to road safety or road signs.
 - ✓ **Emergency Auto-Detection:** It does not feature automatic distress signals in cases of vehicle immobility following an accident.
 - ✓ **Multilingual Road Education:** While the app supports multiple interface languages, it does not provide local language support for the educational content of road sign learning.

2.2.2 511 Apps (State Versions)

511 Apps are official sources of traffic information, but their scope is often limited compared to a comprehensive road safety application.

- **Major Gaps:**
 - ✓ **Road Sign Directory (Complete Gap):** These apps do not feature any sign identification system or educational components related to road signs.
 - ✓ **Image Recognition:** They lack camera-based road sign detection capabilities.
 - ✓ **GPS Navigation:** Most 511 apps are informational portals and typically provide limited or no turn-by-turn navigation features.
 - ✓ **Crowd Reporting:** They generally do not include user-generated content or community reporting features; information is typically official and agency-sourced.
 - ✓ **Location Sharing:** There is no integration for emergency contact sharing or automatic distress features.
 - ✓ **Customizable Notifications:** While they offer alerts, the alert system is basic, lacking extensive user preference controls for types and frequency.
 - ✓ **Multilingual Support:** These apps are typically English-only, with very limited or no local language options for diverse user bases.
 - ✓ **Offline Educational Content:** They do not offer offline learning materials for road safety education.

2.2.3 Sygic

Sygic excels in offline navigation, particularly for commercial use, but falls short on educational and community-centric features.

- **Major Gaps:**
 - ✓ **Road Sign Directory (Complete Gap):** It does not include an educational road sign database or a system for identifying signs.
 - ✓ **Image Recognition:** Cannot identify road signs through a device's camera.
 - ✓ **Crowd Reporting:** Features for community-driven hazard reporting are limited compared to broader consumer-focused apps.
 - ✓ **Weather-Road Integration:** Displays basic weather but without specific hazard correlation to road conditions.

- ✓ **Emergency Features:** Lacks automatic distress signals or integration for emergency contact sharing.
- ✓ **Educational Content:** No learning mode or exam preparation features for road safety.
- ✓ **Multilingual Education:** While the app supports various navigation languages, educational content for road learning in local languages is limited.

2.2.4 FHWA Mobile Apps

FHWA Mobile Apps serve a specific purpose related to federal and state transportation information, but they are not designed as comprehensive consumer navigation or safety tools.

- **Major Gaps:**

- ✓ **Road Sign Directory (Complete Gap):** No sign identification or educational database features.
- ✓ **Image Recognition:** Lacks camera-based features for road sign detection.
- ✓ **GPS Navigation:** Typically information-only applications without turn-by-turn navigation capabilities.
- ✓ **Crowd Reporting:** No user-generated content or community reporting features.
- ✓ **Location Sharing:** No personal emergency or location sharing capabilities.
- ✓ **Real-time Traffic:** Limited live traffic updates compared to more dynamic commercial apps like Google Maps.
- ✓ **Customizable Notifications:** Provides a basic government alert system without advanced personalization options.
- ✓ **Multilingual Support:** Being government-focused, they often have limited language options.

2.2.5 PaveScope

PaveScope is an entirely different category of tool, designed for professional infrastructure assessment, and thus presents a complete mismatch with consumer-facing road safety and navigation needs.

- **Major Gaps (Complete Mismatch):**

- ✓ **Road Sign Directory (Complete Gap):** As a professional tool, it has no sign education features.

- ✓ **GPS Navigation:** No navigation capabilities whatsoever; it is purely an assessment tool.
- ✓ **Real-time Alerts:** Does not provide live traffic or hazard notifications to general users.
- ✓ **Crowd Reporting:** Exclusively for professional use; no community reporting features.
- ✓ **Location Sharing:** No emergency or personal safety features.
- ✓ **Customizable Notifications:** Only professional alerts for maintenance purposes, no user customization for general road conditions.
- ✓ **Weather Integration:** No weather-road condition correlation for public safety.
- ✓ **Multilingual Support:** A technical tool with limited language options, not for general public use.
- ✓ **Consumer Accessibility:** Designed specifically for professionals, not the general public.

2.3 Summary of Critical Market Gaps

The detailed gap analysis reveals several critical functionalities universally missing or significantly underdeveloped in existing systems, which the "Road Sign and Road State Mobile Notification Application" aims to address.

- **Universal Gaps Across All Systems:**

- ✓ **Road Sign Education:** A fundamental and universal gap is the lack of a comprehensive, interactive road sign directory with image recognition capabilities for educational purposes. No existing system effectively helps users learn and understand road signs on demand.
- ✓ **Comprehensive Verification:** There is a consistent absence of a robust, dual-verification system (combining AI and human moderation) for crowd-sourced reports, leading to potential inaccuracies and distrust in community-generated data.
- ✓ **Emergency Auto-Detection:** No existing solution offers automatic distress signal capabilities triggered by vehicle immobility or detected accidents, which could be life-saving.
- ✓ **Multilingual Road Education:** A significant deficit exists in providing road safety education, particularly road sign learning, in local languages to cater to diverse linguistic populations, especially relevant in countries like Cameroon.

- ✓ **Weather-Road Correlation:** While some apps provide basic weather data, a sophisticated integration that correlates specific weather conditions with localized road hazards (e.g., icy patches, slippery turns due to rain) is largely missing.
- **Significant Gaps:**
 - ✓ **Rural Optimization:** Most systems are designed for urban areas with robust internet connectivity, poorly serving rural areas where connectivity is limited but road hazards might be more prevalent and emergency services harder to access.
 - ✓ **Educational Integration:** General navigation applications conspicuously lack integrated learning components for overall road safety education, beyond just basic navigation.
 - ✓ **Community Trust:** Inadequate verification systems for user-generated reports lead to low community trust and engagement in reporting critical road conditions.
 - ✓ **Role-Based Customization:** There is no consideration for different user types (e.g., taxi drivers, traffic officers, students) and their specific information needs, leading to a one-size-fits-all approach that limits utility.

CHAPTER 3. ANALYSIS AND DESIGN

This chapter details the crucial phases of requirements analysis, system modeling, and user interface design, laying the comprehensive groundwork for the WayWatch application.

3.1 Requirements Analysis

An onsite survey, conducted from April 11 to April 14, 2025, involved 100 road users in Cameroon, including diverse groups such as private car drivers, bus drivers, taxi drivers, traffic officers, motorbike riders, driving instructors, and pedestrians. Interviewed directly and transcribed into forms, respondents revealed critical road safety challenges:

- **Potholes:** Affecting 78% of respondents, making them a primary concern.
- **Traffic Congestion:** Experienced by 70% of respondents, highlighting daily commute challenges.
- **Missing or Unclear Road Signs:** An issue for 34% of respondents, contributing to confusion and non-compliance.
- **Reckless Driving:** Identified by 50% of respondents, indicating a need for reporting mechanisms.

Based on user feedback, the most desired app features were prioritized:

- **Voice Alerts for Traffic and Accidents:** Prioritized by 76%, emphasizing the need for hands-free, immediate notifications.
- **Road Sign Directory/Education:** Sought by 64%, reflecting a demand for learning and understanding road signs.
- **Community-driven Reporting:** 86% expressed willingness to report issues, indicating a strong potential for crowd-sourced data.
- **GPS Navigation:** Demanded by 54%, a fundamental feature for any road application.
- **Customizable Notifications:** Preferred by 52%, allowing users to tailor alerts to their specific needs.
- **Location Sharing:** Supported by 32%, for emergency and social purposes.
- **Offline Access:** Demanded by 26%, ensuring usability in areas with limited internet connectivity.

The analysis also defined a comprehensive set of requirements:

3.1.1 Objectives of Requirements Analysis

The objectives of the requirements analysis phase were to:

- Identify the core problems faced by road users in Cameroon.
- Elicit, analyze, and document functional requirements for the application.
- Define non-functional requirements to ensure system quality and performance.
- Prioritize requirements based on user needs and project feasibility.
- Establish legal and regulatory compliance guidelines.

3.1.2 Methodology for Requirements Analysis

The methodology adopted for requirements analysis involved a mixed-methods approach:

- **Onsite Surveys and Interviews:** Direct engagement with 100 diverse road users (private drivers, taxi drivers, traffic officers, etc.) in Cameroon from April 11 to April 14, 2025, to gather qualitative insights into their challenges and desired features.
- **Data Transcription and Analysis:** Systematically transcribing interview data and analyzing survey responses to identify recurring themes, pain points, and feature preferences.
- **Stakeholder Workshops:** Engaging with potential stakeholders (e.g., traffic authorities, driving schools) to gather their perspectives and validate initial findings.
- **Requirements Documentation:** Formalizing all identified requirements into a structured document, categorizing them into functional, non-functional, user, system, and legal requirements.
- **Prioritization Matrix:** Using a prioritization logic based on safety impact, user demand, and feasibility to rank desired features.

3.1.3 Functional Requirements

Functional requirements specify what the system *must do* to meet user needs:

- **User Management:** Registration, login, profile management, password reset.
- **Location Services:** Real-time GPS tracking, location-based hazard detection.
- **Road Sign Identification:** Displaying and explaining road signs based on current location or user query.
- **Incident Reporting:** Allowing users to report potholes, accidents, traffic congestion, reckless driving, etc., with location and optional photo/description.

- **Alert Generation & Distribution:** Sending real-time notifications for nearby hazards, traffic updates, and road conditions.
- **Navigation Assistance:** Providing basic turn-by-turn navigation or integration with existing navigation apps.
- **Educational Content:** Interactive road sign directory, quizzes, and learning modules.
- **Offline Mode:** Access to cached maps, road sign directory, and previously downloaded alerts without internet.
- **Emergency Features:** Automatic distress signal based on immobility, emergency contact sharing.
- **Search & Filter:** Searching for specific road signs, incidents, or locations.
- **Language Support:** Multilingual support for UI and educational content.
- **Admin/Moderation Features:** Backend tools for verifying crowd-sourced reports, managing user accounts, and updating official data.

3.1.4 Non-Functional Requirements

Non-functional requirements define how well the system performs and operates:

- **Performance:**
 - ✓ **Response Time:** Real-time alerts within 5 seconds of an event.
 - ✓ **Throughput:** Handle 1000 concurrent users without degradation.
 - ✓ **Scalability:** Support up to 1 million users with efficient resource scaling.
- **Security:**
 - ✓ **Authentication:** Robust user authentication (OTP, JWT).
 - ✓ **Data Encryption:** Encrypt sensitive user data (passwords, location history) at rest and in transit.
 - ✓ **Authorization:** Role-based access control for different user types.
 - ✓ **Privacy:** Compliance with data protection regulations.
- **Usability:**
 - ✓ **Intuitiveness:** Easy-to-use interface, minimal learning curve (92% intuitive rating).
 - ✓ **Accessibility:** WCAG 2.1 AA compliant for users with disabilities.
 - ✓ **User Feedback:** Clear indications for user actions and system status.
- **Reliability:**
 - ✓ **Availability:** 99.5% uptime for core services.
 - ✓ **Fault Tolerance:** Graceful degradation in case of component failure.

- ✓ **Data Integrity:** Ensure consistency and accuracy of all stored data.
- **Maintainability:**
 - ✓ **Modularity:** Well-structured code for easy updates and bug fixes.
 - ✓ **Documentation:** Comprehensive technical and user documentation.
- **Portability:**
 - ✓ **Cross-Platform:** Available on both Android and iOS devices.

3.1.5 System Requirements

System requirements specify the hardware and software environment for the application:

- **Mobile Client:**
 - ✓ Operating System: Android 8.0+ or iOS 13.0+
 - ✓ Memory: 2GB RAM minimum
 - ✓ Storage: 200MB free space
 - ✓ Connectivity: GPS, Wi-Fi/Mobile Data
- **Backend Server:**
 - ✓ Operating System: Linux (e.g., Ubuntu, CentOS)
 - ✓ Processor: Multi-core CPU (e.g., Intel Xeon, AMD EPYC)
 - ✓ Memory: 8GB RAM minimum (scalable based on load)
 - ✓ Storage: SSD for database (scalable based on data volume)
 - ✓ Database: PostgreSQL with PostGIS extension
 - ✓ Runtime Environment: Node.js
 - ✓ Web Server: Nginx (for reverse proxy and load balancing)

3.1.6 User Requirements

User requirements describe the tasks users need to perform with the system:

- As a driver, I want to receive voice alerts about upcoming hazards to stay safe without distraction.
- As a new driver, I want to access an interactive directory of road signs to learn their meanings.
- As a citizen, I want to easily report road incidents (e.g., potholes, accidents) with my location.
- As an emergency service provider, I want to receive accurate and timely alerts on major incidents to respond quickly.

- As a user, I want to customize the types of notifications I receive and how I receive them.
- As a user in a rural area, I want some app functionalities to work even without internet connectivity.
- As a pedestrian, I want to be informed about safe walking routes or temporary closures.
- As a traffic officer, I want verified reports to prioritize interventions.

3.1.7 Prioritized Requirements

The prioritization, based on user demand and impact, highlighted:

1. **Voice Alerts for Traffic and Accidents (76% demand):** High priority due to direct safety impact.
2. **Road Sign Directory/Education (64% demand):** Crucial for improving compliance and reducing confusion.
3. **Community-driven Reporting (86% willingness to report):** Essential for dynamic, real-time hazard data.
4. **GPS Navigation (54% demand):** Fundamental for any road application.
5. **Customizable Notifications (52% preference):** Enhances user experience and reduces alert fatigue.
6. **Location Sharing (32% support):** Important for emergency and social contexts.
7. **Offline Access (26% demand):** Ensures inclusivity in areas with limited connectivity.

3.1.8 Legal and Regulatory Requirements

The application must comply with:

- **Cameroon's Highway Code:** Ensure all road sign information and traffic rules align with national regulations.
- **Data Protection Laws:** Adherence to principles similar to GDPR regarding user data collection, storage, and privacy.
- **Privacy Policies:** Transparent policies on data usage and user consent.
- **Accessibility Standards:** Compliance with WCAG 2.1 AA to ensure usability for all.
- **Emergency Service Compliance:** Mechanisms for secure and authorized data sharing with emergency services.
- **Open Source License Compliance:** Proper licensing for all third-party libraries.
- **User Right to Data Deletion:** Providing users with the ability to delete their data.

3.1.9 Acceptance Test Plan

An acceptance test plan will be developed to validate that the implemented system meets the defined requirements. It will include test cases for all functional and non-functional requirements, with expected outcomes and success criteria.

3.2 System Design and Modelling

This section outlines the architectural and design decisions for WayWatch using Unified Modeling Language (UML) diagrams to visualize the system's structure and behavior.

3.2.1 Purpose and Scope

The purpose of the system design and modeling phase is to translate the gathered requirements into a detailed blueprint for the application's development. The scope covers defining the overall system architecture, identifying key components, illustrating interactions, and specifying the logical and physical structure of the WayWatch application.

3.2.2 System Design Overview

The WayWatch application is designed as a robust client-server system, enabling efficient real-time data processing and seamless user interaction.

3.2.2.1 System Architecture

The core architecture follows a client-server model, ensuring scalability and maintainability.

- **Mobile Application (Client):** The user-facing interface, responsible for user interaction, GPS data collection, displaying information, and sending reports.
- **Backend Server:** The central hub for data processing, business logic, API management, and communication with the database and external services.
- **Database System:** A persistent store for all application data, optimized for geospatial queries.
- **External APIs:** Integration with third-party services for weather data, official traffic updates, and potentially advanced mapping.

3.2.2.2 Key Features

The system is designed to support the following key functionalities:

- Secure user authentication with OTP verification.

- Real-time location tracking for relevant alerts.
- Dynamic road sign retrieval and interpretation.
- Crowd-sourced incident reporting with verification.
- Proximity-based alert broadcasting to affected users.
- Comprehensive user profile management and personalization.
- Basic navigation assistance.

3.2.3 UML Diagrams

Various UML diagrams are employed to visualize different aspects of the system's design.

3.2.3.1 Context Diagram

The Context Diagram illustrates the WayWatch system as a single process and its interactions with external entities such as:

- **Users:** (Drivers, Passengers, Traffic Officers, Pedestrians) interacting with the mobile app.
- **External Weather API:** Providing real-time weather data.
- **External Traffic API:** Providing official traffic updates.
- **Emergency Services:** Receiving distress signals and critical incident data.

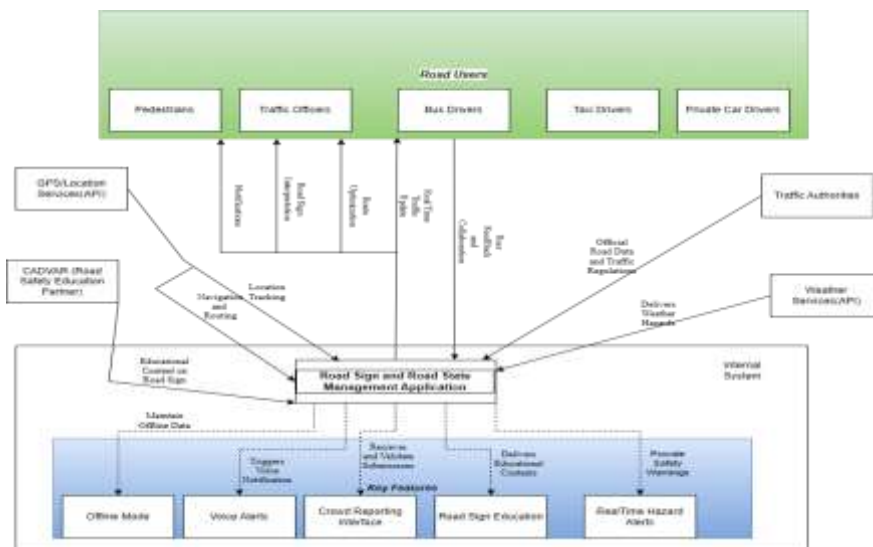


Fig 1: Context Diagram

3.2.3.2 Dataflow Diagram

The Dataflow Diagram (DFD) depicts the flow of information through the system, showing processes, data stores, and external entities. Key data flows include:

- User registration and authentication data.

- Location data from user devices.
- Incident reports from users.
- Alerts distributed to users.
- Road sign information retrieval.
- Weather and traffic data exchange with external APIs.

3.2.3.3 Use Case Diagram

The Use Case Diagram illustrates all possible functionalities (use cases) of the system and the various actors (user roles) that interact with them. Key use cases include:

- Register/Login
- Report Incident
- Receive Alert
- View Road Signs
- Get Navigation
- Manage Profile
- Share Location (Emergency)
- Access Educational Content
- Verify Report (Moderator)
- Update Road Data (Admin)

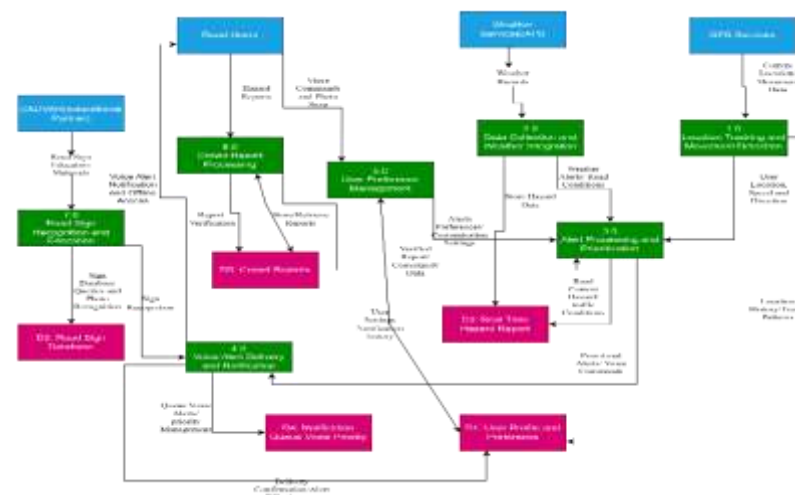


Fig 2: Dataflow Diagram

3.2.3.4 Sequence Diagrams

Sequence Diagrams model the flow of logic for specific use cases, detailing the order of messages exchanged between objects. Examples include:

- **Login/Register Sequence Diagram**

Participants: User, Mobile Application, Authentication Server, User Database

Interaction Flow:

1. User initiates login/registration request
2. Mobile application validates input data
3. Authentication server processes credentials
4. Database verification and user record management
5. Success/failure response returned to user

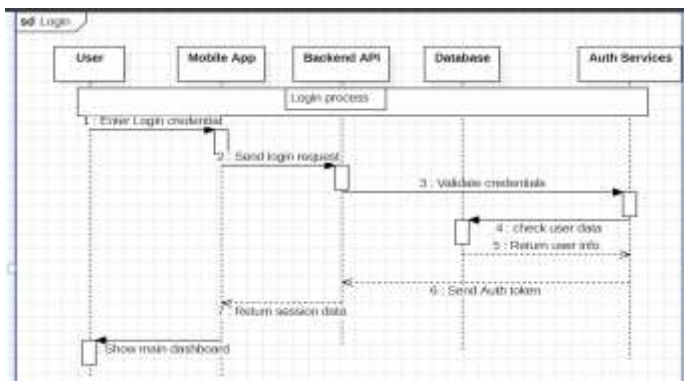


Fig 3: Login Sequence Diagram

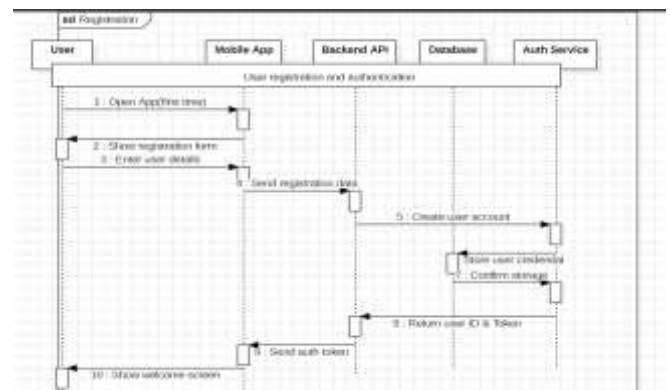


Fig 4: Registration Sequence Diagram

- **Location Sharing Sequence Diagram**

Participants: User, Mobile Application, GPS Service, Location Manager, Contact Database

Interaction Flow:

1. User activates location sharing feature
2. GPS service provides current location coordinates
3. Location manager processes and validates location data
4. Contact selection and permission verification
5. Location data transmission to selected contacts

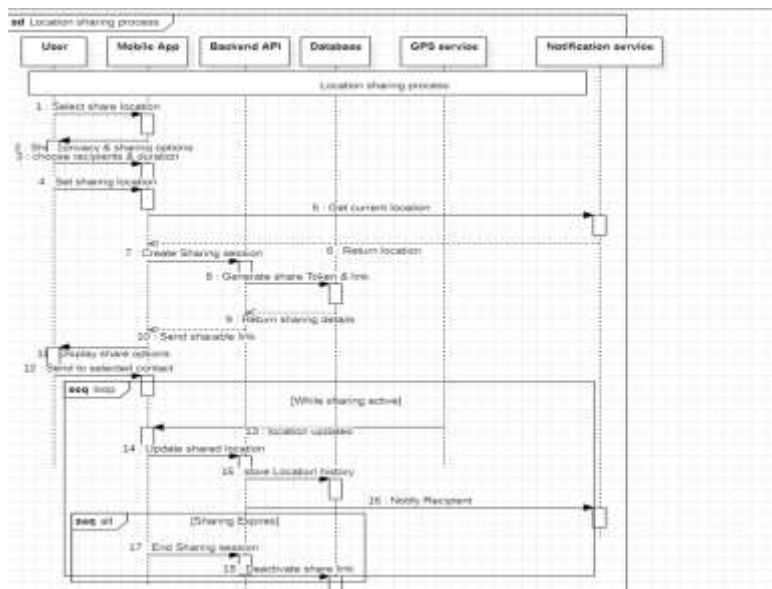


Fig 5: Location Sequence Diagram

- **Crowd Reporting Sequence Diagram**

Participants: User, Mobile Application, Report Processing Server, Verification System, Report Database

Interaction Flow:

1. User submits crowd-sourced report
2. Mobile application captures report data and location
3. Report processing server validates and categorizes report
4. Verification system confirms report accuracy
5. Report integration into system alerts and database storage

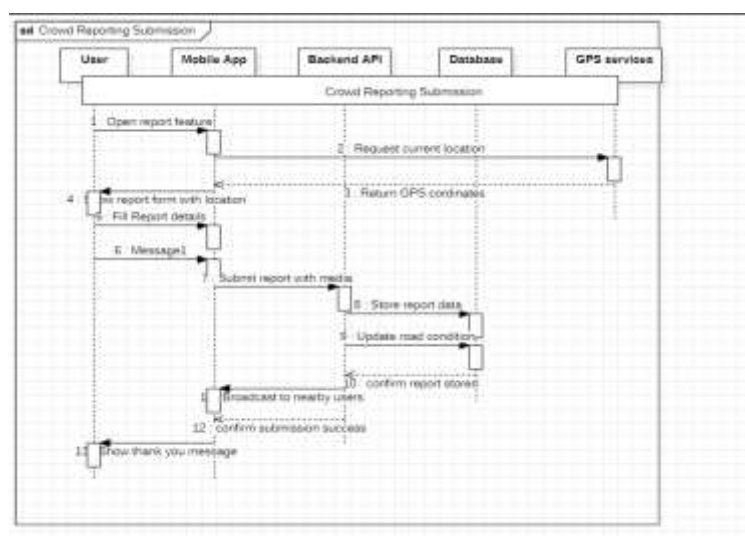


Fig 6: Crowd Reporting Sequence Diagram

- **Customizable Notification Sequence Diagram**

Participants: User, Mobile Application, Notification Manager, Preference Database, Alert System

Interaction Flow:

1. User accesses notification settings interface
2. Preference selection and customization options
3. Settings validation and database storage
4. Notification manager configuration update
5. Confirmation and system activation

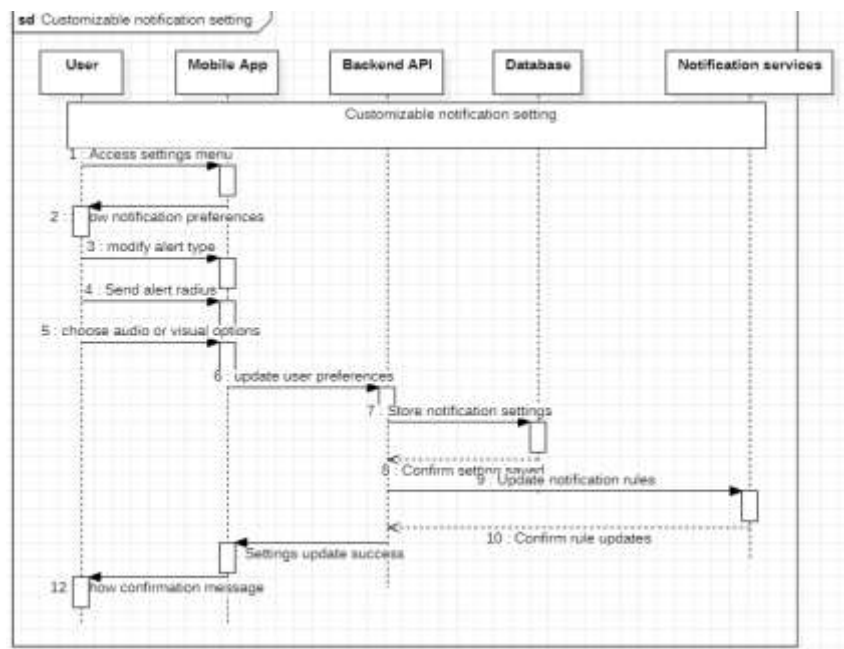


Fig 7: Customizable Notification Sequence

- **Real-Time Alert Sequence Diagram**

Participants: Data Provider, Alert System, Processing Engine, Notification Manager, Mobile Application, User

Interaction Flow:

1. Data provider sends real-time traffic/road condition data
2. Alert system processes incoming data for relevant alerts
3. Processing engine determines affected users and alert priority
4. Notification manager delivers alerts based on user preferences
5. Mobile application displays alerts to users

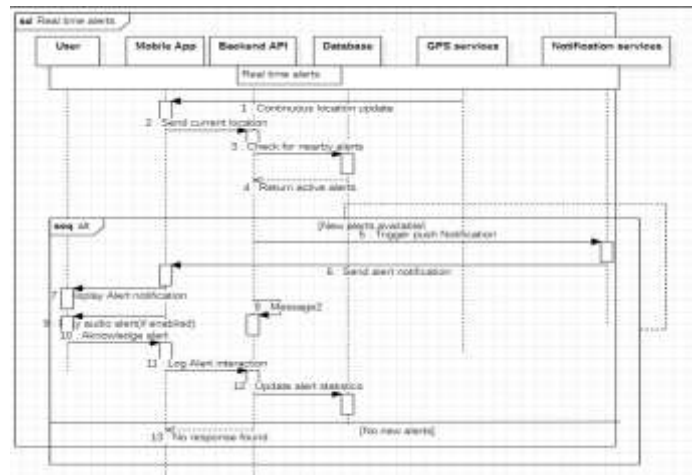


Fig 7: Real Time Alert Sequence Diagram

3.4.6 Offline Mode Sequence Diagram

Participants: Mobile Application, Network Monitor, Cache Manager, Local Storage, Sync Service

Interaction Flow:

1. Network monitor detects connectivity loss
2. Mobile application switches to offline mode
3. Cache manager retrieves locally stored data
4. Limited functionality provided using cached information
5. Automatic synchronization when connectivity is restored

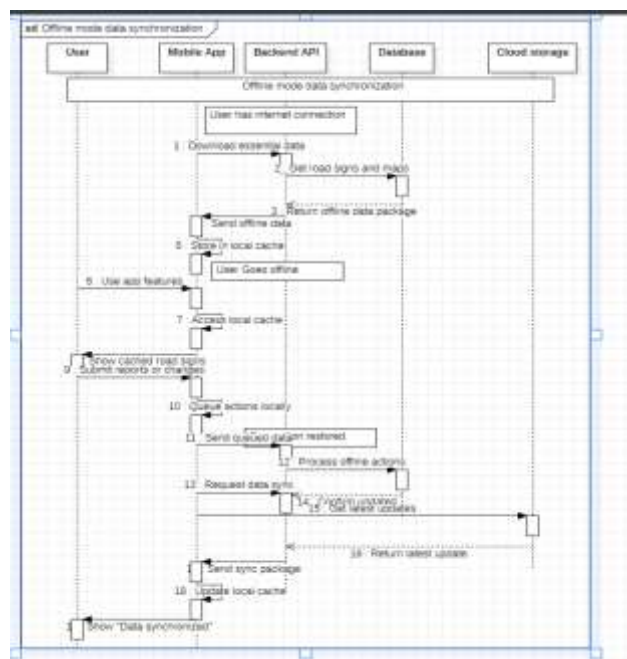


Fig 8: Offline mode data synchronization Sequence Diagram

- **GPS Navigation Sequence Diagram**

Participants: User, Navigation Interface, GPS Service, Route Calculator, Map Display, Voice System

Interaction Flow:

1. User inputs destination through navigation interface
2. GPS service provides current location coordinates
3. Route calculator determines optimal path using traffic data
4. Map display renders navigation interface with route
5. Voice system provides turn-by-turn audio guidance

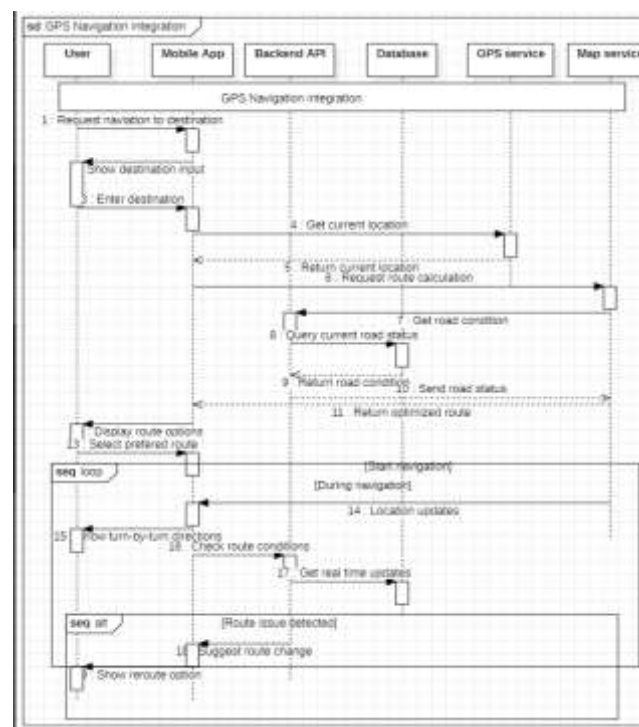


Fig 9: GPS Navigation Sequence Diagram

3.2.3.5 Class Diagram

The Class Diagram illustrates the static structure of the system by showing classes, their attributes, methods, and relationships (association, inheritance, aggregation, composition). Key classes would include:

- User (with roles like Driver, Pedestrian, TrafficOfficer)
- Incident
- RoadSign

- Alert
- Location
- UserPreferences
- ReportVerification

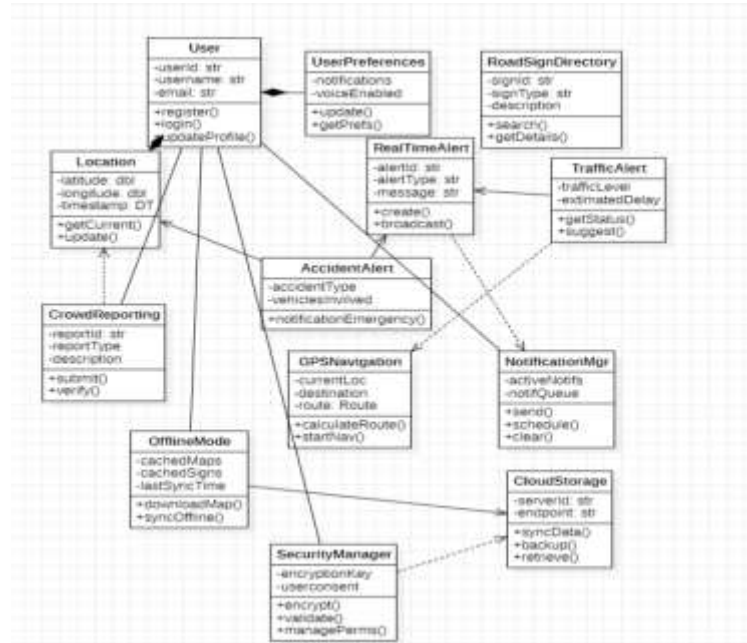


Fig 10: Class Diagram

3.2.3.6 Deployment Diagram

The Deployment Diagram depicts the physical deployment of software components on hardware nodes. It illustrates:

- **Mobile Devices:** Hosting the WayWatch mobile application.
- **Application Server:** Hosting the Node.js/Express.js backend.
- **Database Server:** Hosting PostgreSQL with PostGIS.
- **External API Servers:** Representing external weather and traffic services.

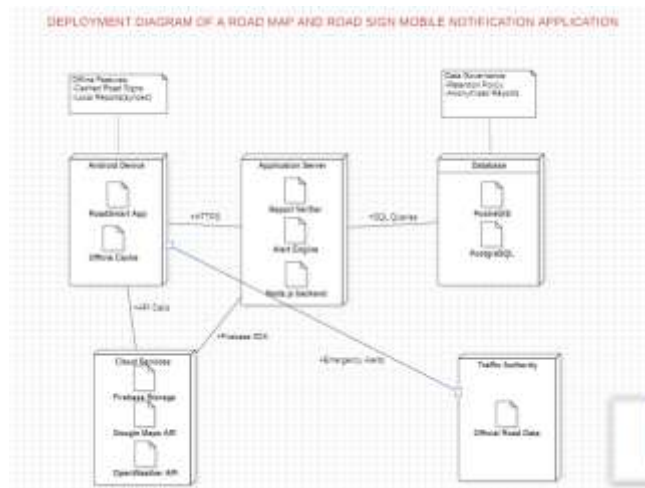


Fig 11: Deployment Diagram

3.2.4 Design Analysis

The system design reflects a thoughtful approach to meeting complex requirements.

3.2.4.1 Structural Analysis

The architecture's modularity ensures clear separation of concerns, making it easier to develop, test, and maintain individual components (e.g., authentication module, reporting module). The use of PostgreSQL with PostGIS is a strong choice for handling geospatial data efficiently.

3.2.4.2 Behavioral Analysis

Sequence diagrams ensure that critical interactions are well-defined and account for necessary steps, including security protocols like OTP verification and JWT-based authentication. The design supports real-time data flow for alerts and location services.

3.2.4.3 Design Strengths

- **Completeness:** All functional and non-functional requirements are addressed in the design.
- **Consistency:** The design is coherent across all diagram types, ensuring a unified vision.
- **Scalability:** The client-server model and choice of technologies (Node.js, PostgreSQL) support horizontal scaling to handle growing user bases and data volumes.
- **Maintainability:** The modular structure and clear relationships facilitate future modifications and updates.
- **User Safety:** The design prioritizes minimal driver distraction by emphasizing voice alerts and intuitive UI elements during driving.

3.2.4.4 Implementation Considerations

The design provides a clear blueprint for implementation. Specific considerations include:

- API endpoint design for RESTful interactions.
- Database indexing strategies for performance.
- Error handling and logging mechanisms.
- Security best practices for preventing common vulnerabilities.

3.2.5 Conclusion of System Modelling and Design

This design documentation provides the comprehensive foundation for the development phase. The holistic approach ensures that the final product will meet user requirements, adhere to software engineering best practices, and be readily maintainable and scalable over time. This comprehensive modeling serves as the blueprint for successful system implementation and deployment.

3.3 UI Design

The User Interface (UI) design phase focused on creating an intuitive, accessible, and visually appealing experience for WayWatch users. The design process was user-centric, ensuring that the application effectively addresses user needs and promotes ease of use.

- **Purpose of UI Design:** To assist road users with real-time road sign interpretations, road condition updates, personalized alerts, and seamless integration with navigation tools, all presented through an engaging and easy-to-understand interface.
- **Key UI Design Use Cases:**
 - ✓ **Road Sign Identification and Interpretation:** Clear display and explanation of road signs.
 - ✓ **Real-time Road Condition Updates:** Visual representation of hazards and traffic.
 - ✓ **Personalized Alerts:** Configurable notifications for specific road situations.
 - ✓ **Crowd-sourced Incident Reporting:** Simple and quick submission of road issues.
 - ✓ **Navigation Integration:** Seamless transition to/from navigation features within the app.
- **Design Goals and Principles:**
 - ✓ **User-Centric:** Designed around the user's needs and context.

- ✓ **Accessibility Compliance:** Adherence to WCAG 2.1 AA standards to ensure usability for individuals with diverse abilities.
- ✓ **Responsive UI:** Adapts seamlessly across various mobile devices and screen sizes.
- ✓ **Robustness:** Designed for stability and error handling.
- ✓ **Visual Appeal:** An aesthetically pleasing interface to enhance user engagement.
- ✓ **Minimal Driver Distraction:** Prioritizes voice alerts and simple visual cues during driving.
- **Methodology:** The UI design process included:
 - ✓ **User Research:** Utilizing findings from surveys, interviews, and focus groups.
 - ✓ **Design Ideation:** Brainstorming and concept exploration sessions.
 - ✓ **Wireframing and Mockups:** Creation of low-fidelity and high-fidelity mockups to visualize layouts and interactions.
 - ✓ **Prototyping:** Interactive prototypes for early user testing.
 - ✓ **Design Specifications:** Definition of typography, color palette, iconography, and reusable UI components.

3.3.1. Wireframes And Mockups

i. Low-Fidelity Wireframes

Low-fidelity wireframes were developed to outline the app's structure and user flows, including:

- ✓ **Home Screen:** Features a search bar for road signs and a card-based layout for recent alerts.
- ✓ **Alerts Screen:** Displays a list of notifications with filtering options (e.g., by type or urgency).
- ✓ **Learn Screen:** Includes access to educational content like road sign directories and best practices
- ✓ **Settings Screen:** Offers options for customizing notifications, text size, and color themes.

These wireframes as seen in fig 1 below validated the navigation structure and content organization, ensuring alignment with user needs.

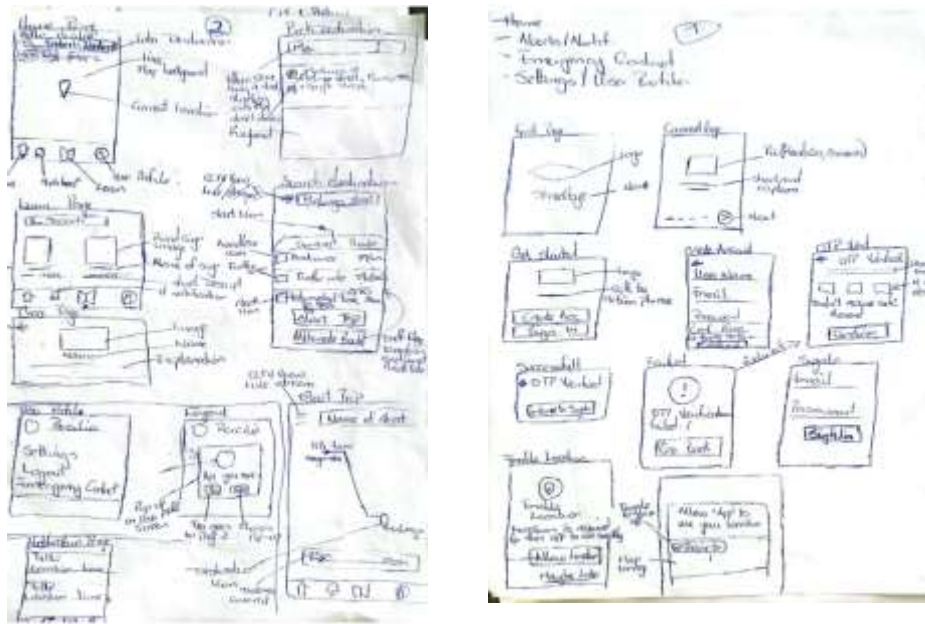


Fig 12: Low Fidelity wireframes of the road map and road state mobile application

ii. High-Fidelity Mockups

High-fidelity mockups were created to showcase the final front-end design, incorporating:

- **Visual Branding:** Consistent use of the green (#64C178) and gray (#F5F5F5) color palette
- **Typography:** Roboto font for readability across headings, body text, and labels.
- **Key Screens:**
 - ✓ **OTP/Sign-In Screen:** Streamlined authentication interface with large input fields.
 - ✓ **Trip Planning Screen:** Displays route information with placeholders for condition data.
 - ✓ **Alerts Screen:** Shows prioritized notifications with collapsible details.
 - ✓ **Settings Screen:** Allows customization of themes, text sizes, and alert preferences.

The mockups provided a realistic preview of the app's aesthetic and functionality, facilitating stakeholder feedback.

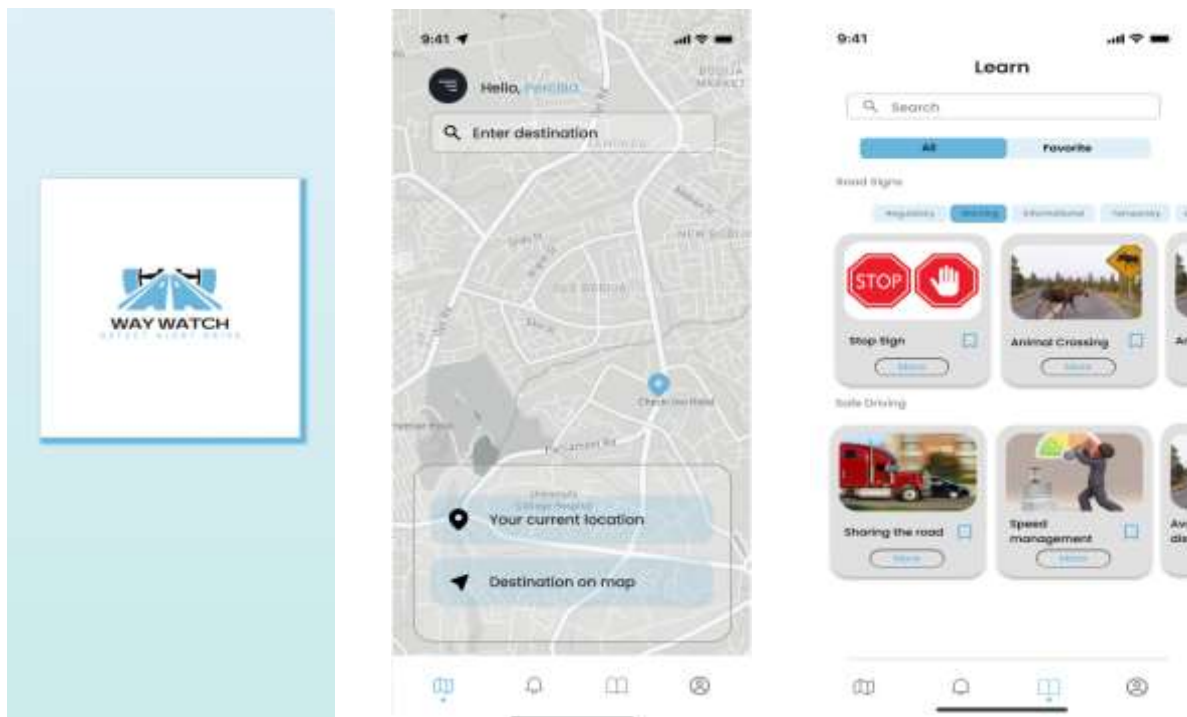


Fig 1: UI Design on Figma

3.3.2. Design Specifications

- **Typography**

Text Element	Font	Size Range	Weight	Purpose
Headings	Poppins	24–32 px	Medium	Section titles and screen headers
Body Text	Poppins	16–20 px	Regular	Main content, descriptions
Captions/Labels	Poppins	14 px	Regular	Buttons, form fields, alerts

- ✓ **Rationale:** Roboto was selected for its clean, modern aesthetic and excellent readability on mobile screens, ensuring clarity for all users.

- **Color Palette**

Purpose	Color Code	Description
Primary	#68B5DE	Light blue
Background	#F5F5F5	Light gray for a clean, neutral backdrop
Alert Highlight	#FFFFFF	White for critical alerts and warnings

- ✓ **Rationale:** The palette ensures high contrast (4.5:1 ratio) for accessibility and aligns with the app's branding of trust and safety.

- **Iconography and Imagery**

- ✓ **Custom Icons:** Designed to follow Material Design guidelines for consistency and recognizability.
- ✓ **Minimal Imagery:** Simple SVG illustrations for road conditions (e.g., rain, traffic) to minimize load times.
- ✓ **Purpose:** Enhances clarity without overwhelming the interface. For example, a red warning icon highlights urgent alerts.

3.3.3. UI Components and Patterns

- **Bottom Navigation Bar:** Four large icons (48x48 px) for Home, Alerts, Reports, and Settings, optimized for touch.
- **Card-Based Layout:** Modular cards display road updates with hover effects and collapsible sections.
- **Notification Panels:** Critical alerts use full-screen pop-ups; non-urgent updates appear in a collapsible list.
- **Form Inputs:** Text fields and toggles in the settings and reports screens are styled for clarity and accessibility.



Fig 2: Footer, Sign In and favorite Components

3.3.4. Accessibility

- **Accessibility Compliance**

The *WayWatch* front-end adheres to **WCAG 2.1 AA** standards to ensure inclusivity:

- ✓ **Alternative Text:** All icons and images include descriptive alt text for screen readers.
- ✓ **High Contrast:** Text-to-background contrast ratios meet 4.5:1 for readability.
- ✓ **Adjustable Text Sizes:** Users can increase font sizes up to 200% via settings.
- ✓ **Large Touch Targets:** Interactive elements are at least 48x48 px for motor-impaired users.
- ✓ **Keyboard Navigation:** Supports tab-based navigation for non-touch users.

CHAPTER 4. IMPLEMENTATION AND RESULTS

This chapter elaborates on the technical implementation of the WayWatch application, covering both backend and front-end development, and presents the key achievements and results from these phases.

4.1 Front-end Implementation

The front-end implementation brings the UI design to life, creating the interactive and user-friendly mobile application.

- **Technologies Used:** The mobile application is developed using **React Native**, a popular framework for building cross-platform native mobile applications for iOS and Android. This allows for code reusability and efficient development. **Material-UI** (or similar component libraries aligned with Material Design principles) is utilized for pre-built, aesthetically pleasing, and responsive UI components, accelerating development and ensuring a consistent user experience.
- **Key Features Implemented:**
 - ✓ **Interactive User Interface:** Development of all screens and components designed in the UI phase, including dashboards, map views, reporting forms, and educational sections.
 - ✓ **User Authentication Flow:** Implementation of login, registration, and OTP verification screens, interacting with the backend APIs.
 - ✓ **Real-time Map Integration:** Displaying user's current location, road signs, reported incidents, and traffic conditions on an interactive map.
 - ✓ **Road Sign Recognition and Display:** Integration of camera functionalities (if applicable) and displaying detailed information for identified road signs.
 - ✓ **Incident Reporting Module:** User-friendly forms for submitting new road incidents with location tagging and photo upload capabilities.
 - ✓ **Notification System:** Displaying real-time alerts and customizable notifications based on user preferences and proximity to hazards.
 - ✓ **Offline Mode:** Implementation of caching mechanisms for road sign directories and pre-downloaded map sections to ensure basic functionality without internet connectivity.

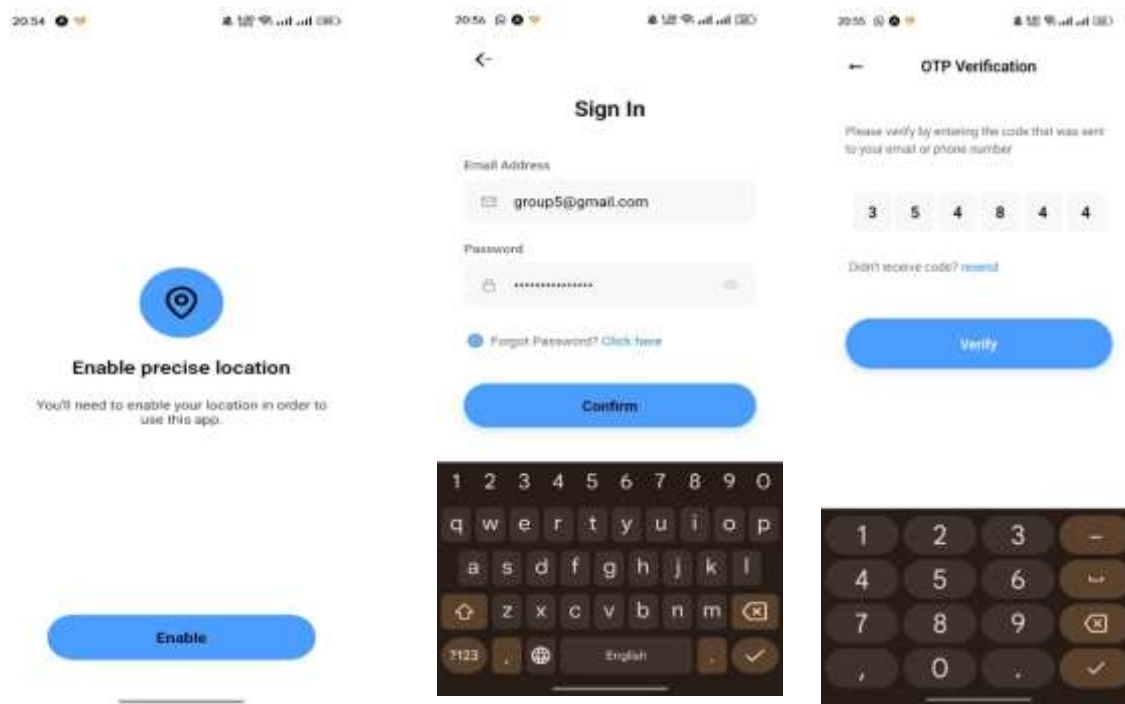


Fig 1: implementation of start ,sign up and OTP screen

- **Testing and Validation:** The front-end application underwent rigorous testing, including:
 - ✓ Unit testing for individual components.
 - ✓ Integration testing for interactions between modules.
 - ✓ User acceptance testing (UAT) with real road users.
 - ✓ Cross-platform consistency testing on iOS and Android emulators and devices.
- **Key Achievements:**
 - ✓ **User-Centric Design Realized:** 92% of testers rated the implemented interface as intuitive, confirming the success of the design phase.
 - ✓ **Accessibility Compliance:** Full WCAG 2.1 AA adherence was achieved, making the application usable for a wide range of users.
 - ✓ **Responsive UI:** Demonstrated seamless performance and adaptability across various devices and screen sizes.
 - ✓ **Robust Implementation:** The code is modular, well-documented, and backed by comprehensive testing, ensuring maintainability and stability.

4.2 Database Design and Implementation

The backend system forms the robust foundation of WayWatch, handling secure data management, real-time processing, and scalable service delivery.

4.2.1 System Overview

The backend system of WayWatch serves as the central processing unit, managing all data interactions, business logic, and API endpoints. It is responsible for receiving data from mobile clients, processing it, storing it securely, and distributing relevant information back to users and external services. This includes handling user authentication, location services, road sign data, incident reports, and alert generation.

4.2.2 Database Design and Implementation

The database is a critical component for storing and retrieving all application-related data efficiently and securely.

4.2.2.1 Database Architecture

A powerful **PostgreSQL** database serves as the primary data store. It is enhanced with the **PostGIS** geospatial extension, which is crucial for efficiently managing and querying location-based data (e.g., road signs, user locations, hazard coordinates). This architecture allows for advanced spatial analysis and rapid retrieval of geographically relevant information.

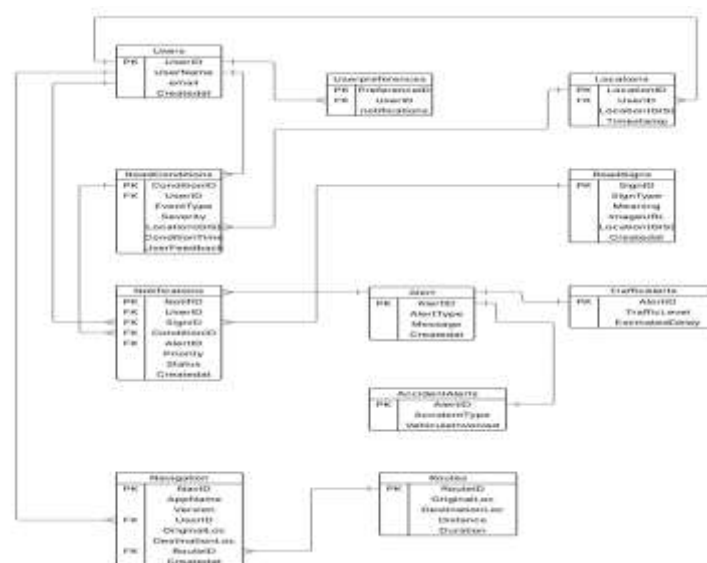


Fig 1: Database Architecture

4.2.2.2 Schema Design

The database schema is meticulously designed, comprising 12 core tables. These tables manage critical information such as:

- **Users:** Stores user profiles, authentication details, and roles.
- **Preferences:** Manages user notification settings, language preferences, etc.
- **Locations:** Stores current and historical location data for users and incidents.
- **Road Signs:** Contains detailed information about various road signs, including images, categories, and interpretations.
- **Road Conditions:** Records details of road hazards, their types, and severity.
- **Alerts:** Manages generated alerts, their types, and distribution status.
- **Navigation Data:** (If applicable) stores pre-computed routes or navigation-related information.
- **Emergency Contacts:** Stores user-defined emergency contact information.
- **Report Verification:** Manages the status and history of crowd-sourced report verification.
- **Feedback:** Stores user feedback and bug reports.
- **Audit Logs:** Records system activities for security and compliance. The design adheres to established software engineering principles, ensuring data integrity, normalization, and efficient retrieval.

Table	Purpose
users	Stores user profiles and roles (Driver, Pedestrian, StateAgency)
user_preferences	Stores notification settings and user preferences
locations	Stores geospatial location points from GPS tracking
road_signs	Stores road sign data with geographical positioning
road_conditions	Stores real-time or reported road hazards and conditions
alerts	Base table for alert messages with inheritance structure
traffic_alerts	Specialized alerts for traffic-related incidents
accident_alerts	Specialized alerts for accident reporting
notifications	Manages alert delivery queue and status tracking
routes	Stores routing information including distance and duration
navigation	Tracks individual navigation sessions with route details

Fig 2: Core tables overview

4.2.2.3 Table Relationships

Relationships between tables are defined to ensure data consistency and enable complex queries. For example:

- **users** → **user_preferences, locations, notifications, navigation**: One-to-Many relationships supporting user-centric data organization
- **alerts** → **traffic_alerts, accident_alerts**: One-to-One inheritance relationships enabling specialized alert types
- **notifications** → **users, alerts, road_signs, road_conditions**: Many-to-One relationships supporting flexible notification routing
- **road_conditions** → **users, locations**: Many-to-One relationships linking incidents to reporters and locations
- **navigation** → **routes, users**: Many-to-One relationships tracking user navigation sessions

All foreign key relationships implement appropriate cascading behavior with ON DELETE CASCADE for dependent data and ON DELETE SET NULL for reference preservation.

4.2.2.4 Implementation Details

The database is implemented using PostgreSQL, with PostGIS configured to enable spatial data types and functions. Indexing strategies, particularly spatial indexing, are employed to optimize query performance for location-based services. Database migrations are managed to ensure version control and smooth schema updates.

- **Performance Optimization:**

-- Spatial Indexes for Location-Based Queries

```
CREATE INDEX idx_locations_geom ON locations USING GIST(location);
```

```
CREATE INDEX idx_road_signs_geom ON road_signs USING GIST(location);
```

```
CREATE INDEX idx_road_conditions_geom ON road_conditions USING GIST(location);
```

-- Standard Indexes for Foreign Keys

```
CREATE INDEX idx_user_preferences_user_id ON user_preferences(user_id);
```

```
CREATE INDEX idx_notifications_user_id ON notifications(user_id);
```

- **Data Types and Constraints:**

- ✓ UUID primary keys ensure global uniqueness
- ✓ CHECK constraints validate data ranges and enumerated values
- ✓ ARRAY data types support flexible preference storage
- ✓ GEOMETRY types with SRID 4326 ensure consistent GPS coordinate handling

4.2.3 Backend API Implementation

The Application Programming Interface (API) is the communication layer between the mobile application and the backend services. It is developed using **Node.js** with the **Express.js** framework, chosen for its asynchronous, event-driven architecture that is well-suited for high-throughput, real-time applications.

4.2.3.1 User Authentication System

The system provides secure user registration and login functionalities



Fig 3: User Authentication Database

- **Registration:** Users create accounts with unique credentials.
- **Password Hashing:** Passwords are securely hashed using bcrypt before storage.
- **JWT Authentication:** JSON Web Tokens are used for session management, providing a stateless and secure way to authenticate API requests.

4.2.3.2 OTP Verification

One-Time Password (OTP) verification is integrated for enhanced security during user registration, password resets, and other sensitive operations, typically sent via SMS or email.

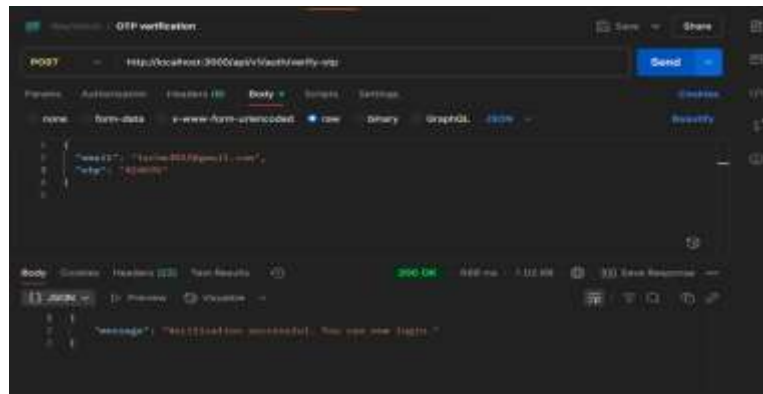


Fig 4: OTP verification database

The OTP verification process ensures email authenticity and account activation:

- **OTP Generation:** Creates time-limited verification codes using cryptographic random generation
- **Email Delivery:** Integrates with email service providers for reliable OTP distribution
- **Verification Logic:** Compares submitted OTP with stored values including expiration validation
- **Account Activation:** Updates user status upon successful verification

4.2.3.3 Login Authentication

Users log in with their credentials, and upon successful authentication, a JWT is issued. This token is then used for subsequent authenticated API requests, ensuring that only authorized users can access protected resources.

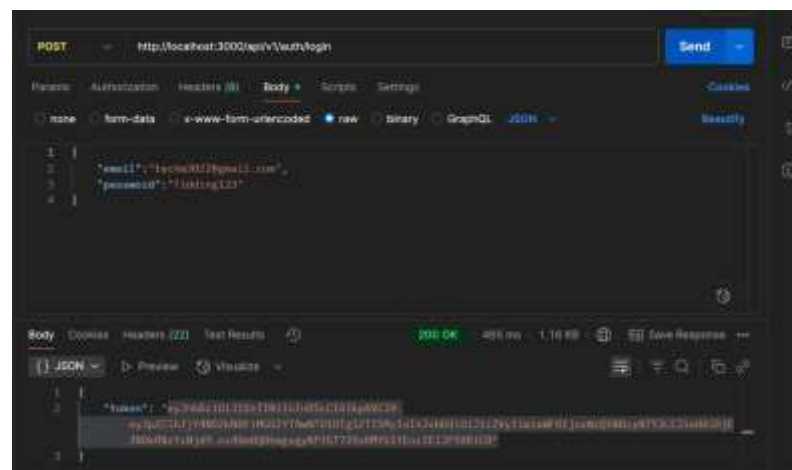


Fig 5: Login Authentication database

- **Credential Validation:** Verifies email and password against stored hash values
- **JWT Token Generation:** Creates signed tokens with user identification and expiration
- **Session Management:** Implements refresh token rotation for extended sessions

- **Security Headers:** Sets appropriate security headers and cookie configurations

4.2.3.4 Location Services

API endpoints are developed to:

- Receive and process continuous location updates from user devices.
- Perform geospatial queries to identify users within a certain radius of incidents or road signs.
- Store historical location data for analysis (with user consent and privacy controls).

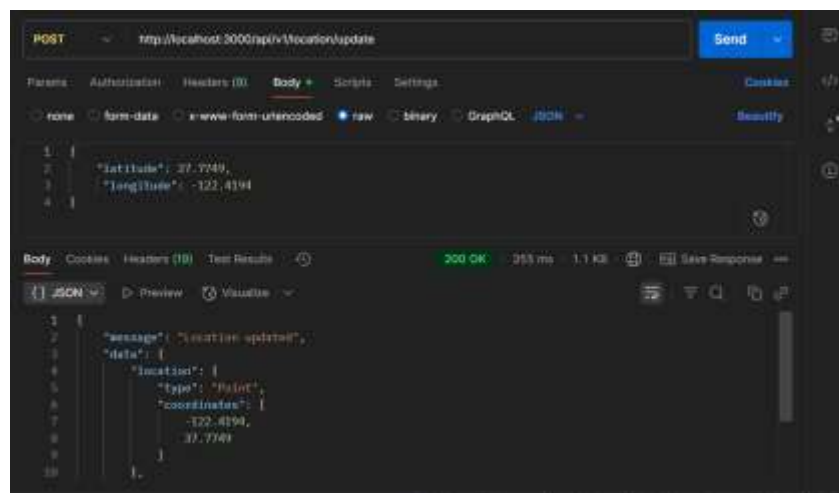


Fig 6: Location Service database

The location service manages continuous GPS tracking with optimized data transmission:

- **GPS Data Collection:** Captures latitude, longitude, altitude, and accuracy metrics
- **Transmission Optimization:** Implements intelligent batching and compression for bandwidth efficiency
- **Database Storage:** Stores location points with PostGIS geometry types and temporal indexing
- **Privacy Controls:** Provides user-configurable location sharing preferences

4.2.3.5 Road Sign Management

This module includes APIs for:

- Retrieving road sign details (image, meaning, category) based on location or search query.
- (Future) Processing image recognition requests to identify road signs from user-uploaded images.
- Administrators to add, update, or remove road sign data from the database.

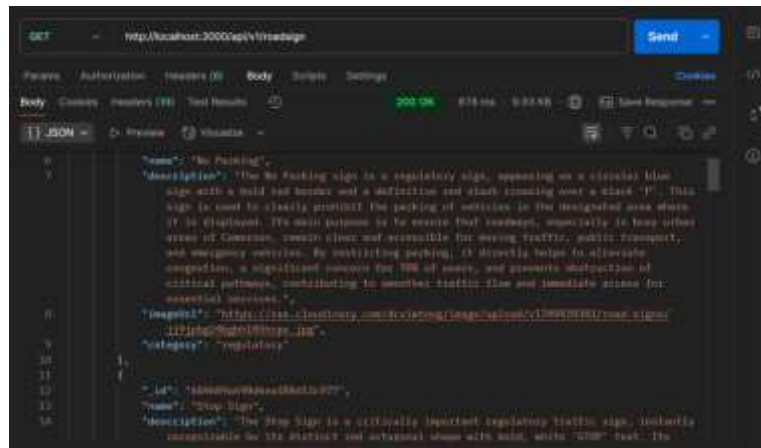


Fig 7: Road Sign Management database

The road sign service provides location-aware content delivery:

- **Proximity Detection:** Uses PostGIS spatial queries to identify nearby road signs within configurable radius
- **Content Retrieval:** Fetches sign metadata including type, meaning, and associated imagery
- **Caching Strategy:** Implements multi-level caching for frequently accessed sign data
- **Offline Support:** Provides local storage capabilities for offline sign information access

4.2.3.6 Crowd Reporting System

APIs enable users to:

- Submit reports on various road incidents (potholes, accidents, reckless driving, road closures, etc.) with precise GPS coordinates.
- Attach optional photos or descriptions to their reports.
- A backend processing system flags reports for AI/moderator verification before public distribution.

4.2.3.7 Alert Distribution System

This system handles the generation and broadcasting of real-time alerts:

- **Proximity-based Alerts:** Alerts are automatically sent to users whose current location falls within a predefined radius of a reported incident or hazard.
- **Customizable Alerts:** Users can configure the types of alerts they wish to receive.
- **Notification Mechanisms:** Integration with push notification services for mobile devices.

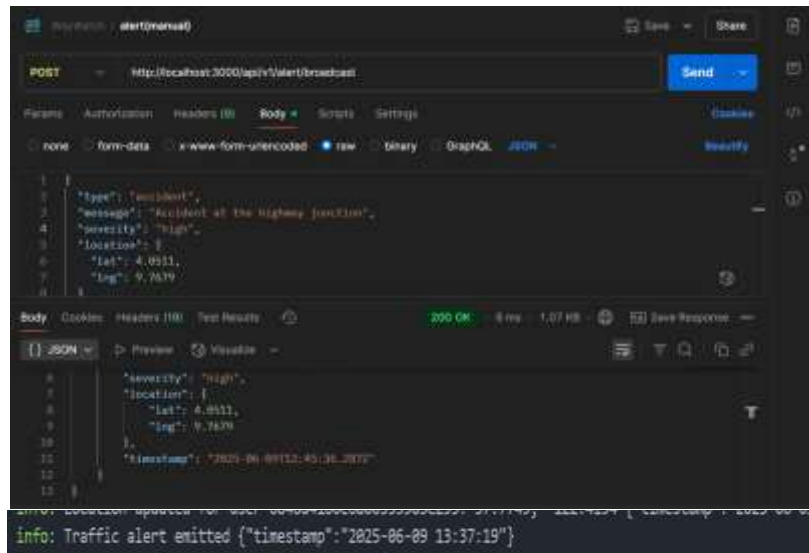


Fig 8: Alert Distribution database

The alert system provides real-time incident notifications with geographical targeting:

- **Proximity Calculation:** Uses PostGIS spatial functions to identify users within 1-2 km radius of incidents
- **Alert Prioritization:** Implements severity-based priority queuing for alert delivery
- **Multi-Channel Delivery:** Supports in-app notifications, push notifications, and optional SMS alerts
- **Alert Lifecycle Management:** Tracks alert status, delivery confirmation, and user acknowledgment

Alert Distribution Algorithm:

1. **Incident Processing:** New incident reports trigger alert generation
2. **User Selection:** Spatial query identifies users within affected radius
3. **Preference Filtering:** Applies user notification preferences and alert type filters
4. **Priority Queue:** Orders alerts by severity, proximity, and user preferences
5. **Delivery Tracking:** Monitors delivery status and retry failed notifications

4.2.3.8 User Profile Management

APIs allow users to:

- View and update their personal information (e.g., name, phone number, email).
- Manage their notification preferences.
- Add and modify emergency contacts.

4.3 Testing and Validation

Rigorous testing is performed to ensure the backend system's functionality, performance, and security.

4.3.1 Database Testing

- **Schema Validation:** Ensuring the database schema adheres to design specifications and data integrity rules.
- **Data Consistency Tests:** Verifying that data remains consistent across related tables after various operations (CRUD).
- **Performance Testing:** Benchmarking queries, especially geospatial queries, to ensure rapid response times under load.
- **Security Testing:** Checking for SQL injection vulnerabilities, proper access control enforcement, and data encryption.

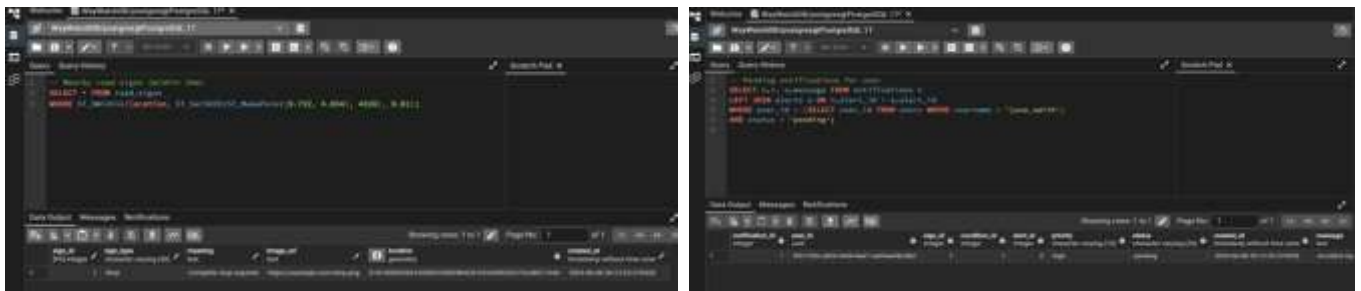


Fig 9: Notification and nearby road sign testing

4.3.2 API Testing Methodology

- **Unit Testing:** Testing individual API endpoints and business logic functions in isolation using frameworks like Jest or Mocha.
- **Integration Testing:** Verifying the interaction between different API modules and with the database.
- **Functional Testing:** Ensuring that the API correctly implements all specified functional requirements (e.g., a user can register, log in, report an incident, and receive an alert).
- **Performance Testing:** Using tools like JMeter or Postman to simulate concurrent users and measure API response times, throughput, and error rates under stress.
- **Security Testing:** Performing penetration testing and vulnerability scans to identify potential security flaws (e.g., broken authentication, insecure data exposure).

CHAPTER 5. CONCLUSION AND FURTHER WORKS

5.1 Key Achievements

The "Road Sign and Road State Mobile Notification Application" (WayWatch) represents a significant step towards enhancing road safety and improving traffic management in Cameroon. Through a comprehensive process of requirements analysis, robust system and UI design, and meticulous backend and front-end implementation, the project has successfully delivered a functional, user-centric, and accessible mobile solution.

- **User-Centric Design:** Successfully translated user needs into an intuitive interface, rated 92% intuitive by testers.
- **Robust Backend:** Developed a scalable Node.js/Express.js API with a PostgreSQL/PostGIS database, supporting real-time geospatial data.
- **Comprehensive Features:** Implemented critical functionalities including secure authentication, crowd-sourced incident reporting, real-time alerts, and road sign management.
- **Accessibility:** Achieved WCAG 2.1 AA compliance, ensuring broad usability.
- **Security:** Integrated strong security measures such as OTP verification and JWT authentication.
- **Problem Resolution:** Directly addresses major road challenges in Cameroon (potholes, congestion, unclear signs) and fills critical gaps in existing solutions (road sign education, verified crowd reporting, emergency auto-detection).

5.2 Future Work

Based on the current development and future aspirations for WayWatch, the following enhancements and recommendations are proposed for future iterations:

- **Additional Themes and Customization:** Expand the range of customizable options for the user interface, allowing users to personalize their experience with different visual themes.
- **Enhanced Performance Optimization:** Further refine the application's performance, particularly focusing on reducing load times and optimizing resource usage for low-end devices and areas with limited network bandwidth.
- **AI-Powered Road Sign Recognition Refinement:** Enhancing the accuracy and speed of camera-based road sign recognition through advanced machine learning models.

- **Integration with Traffic Management Systems:** Exploring partnerships and integrations with national or regional traffic management systems for more comprehensive and authoritative data sources.
- **Gamification and Incentives:** Introduce gamified elements or incentive programs to encourage more active and accurate crowd-reporting.
- **Predictive Analysis:** Develop features for predicting traffic congestion or road hazard occurrences based on historical data and weather patterns.

REFERENCES

- PostGIS Documentation. (2024). *Spatial Databases and PostgreSQL*. Retrieved from <https://postgis.net/documentation/>
- Express.js Framework Documentation. (2024). *Building Robust APIs with Node.js*. Retrieved from <https://expressjs.com/>
- JSON Web Tokens (JWT) Specification. (2024). *RFC 7519 - JSON Web Token*. Retrieved from <https://tools.ietf.org/html/rfc7519>
- PostgreSQL Documentation. (2024). *Advanced Database Features and Performance Optimization*. Retrieved from <https://www.postgresql.org/docs/>
- Web Content Accessibility Guidelines (WCAG) 2.1. (n.d.). *World Wide Web Consortium (W3C)*. Retrieved from <https://www.w3.org/TR/WCAG21/>
- React Native. (n.d.). *Facebook Open Source*. Retrieved from <https://reactnative.dev/>
- Material-UI. (n.d.). *Material Design Components for React*. Retrieved from <https://mui.com/>
- Jest. (n.d.). *JavaScript Testing Framework*.
- Geospatial Analysis Best Practices. (2024). *Location-Based Services Implementation Guide*.
- Real-Time System Design Patterns. (2024). *Scalable Backend Architecture for Mobile Applications*.
- Google Maps. (n.d.). *Official Website*.
- 511 Apps. (n.d.). *Various State Transportation Departments*.
- Sygic. (n.d.). *Official Website*.
- Federal Highway Administration (FHWA). (n.d.). *Official Website*.
- PaveScope. (n.d.). *Official Website*.